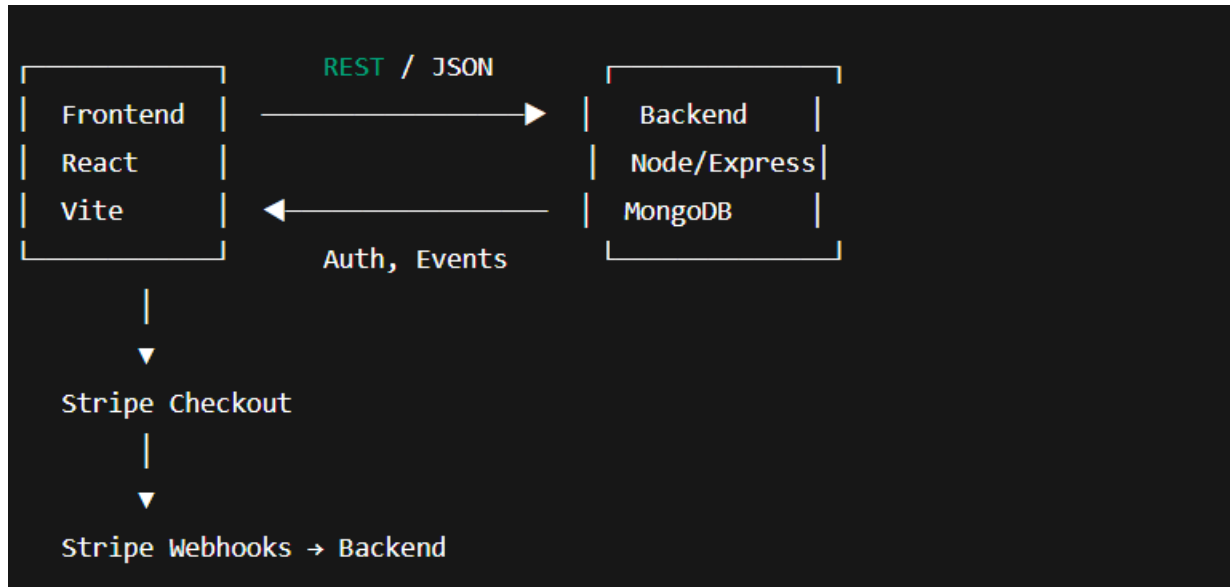


Event Platform Project Architecture



Core responsibilities

- **Frontend:** UI, routing, state, user interactions
- **Backend:** Authentication, business logic, database, payments
- **Shared:** (future) common contracts, types, constants
- **Docs:** API + setup documentation
- **Docker:** Local/dev orchestration

2 Root Folder

```
.
├── backend/
├── frontend/
├── shared/
├── docs/
├── docker-compose.yml
└── README.md
```

docker-compose.yml

- Spins up services together (typically):
 - Backend API
 - Database (MongoDB)
- Makes local development reproducible

docs/

- API.md → REST endpoints, payloads, responses
- SETUP.md → How to run the system locally

shared/

- Reserved for **shared contracts**
 - API types
 - Enums
 - Validation schemas
 - Useful if you later add **mobile apps** or **TypeScript contracts**
-

3 Backend Architecture (Node.js + Express)

This backend follows **Clean Architecture + MVC + Service patterns**.

backend/

```
├── server.js
├── src/
│   ├── app.js
│   ├── config/
│   ├── controllers/
│   ├── models/
│   └── routes/
```

- | └─middlewares/
- | └─validators/
- | └─utils/
- | └─webhooks/
- | └─jobs/
- | └─services/

◆ Entry Layer

server.js

- **Bootstraps the server**
- Starts Express
- Listens on a port
- Connects logging + graceful shutdown

src/app.js

- **Express app configuration**
 - Registers:
 - Middlewares
 - Routes
 - Error handling
 - Keeps server.js clean
-

◆ Configuration Layer (config/)

config/

- └─db.js
- └─env.js

└─ stripe.js

env.js

- Loads environment variables
- Centralizes config access

db.js

- MongoDB connection logic
- Ensures DB is ready before serving requests

stripe.js

- Stripe SDK initialization
 - API keys
 - Webhook secrets
-

◆ **Routing Layer (routes/)**

Routes define **WHAT endpoints exist**, not how they work.

routes/

└─ auth.routes.js

└─ event.routes.js

└─ order.routes.js

└─ ticket.routes.js

└─ webhook.routes.js

Example:

POST /api/events

GET /api/events/:id

POST /api/orders

Each



Calls

a

route:

controller

 Applies auth / validation middleware

◆ **Controller Layer (controllers/)**

Controllers define **request → response logic**.

controllers/

|— auth.controller.js

|— event.controller.js

|— order.controller.js

|— ticket.controller.js

Responsibilities:

- Parse request data
- Call models / services
- Return HTTP responses

Example:

EventController

- createEvent
- updateEvent
- listEvents
- getEventDetails



Controllers should stay **thin** — no heavy business logic.

◆ **Model Layer (models/)**

Mongoose schemas = **database structure + relationships**.

models/

└─ User.model.js
└─ Event.model.js
└─ TicketType.model.js
└─ Ticket.model.js
└─ Order.model.js
└─ Payment.model.js
└─ CheckIn.model.js
└─ Category.model.js

Key domain concepts:

- **User** → organizer or attendee
- **Event** → created by organizer
- **TicketType** → VIP, Regular, Free
- **Ticket** → issued per attendee
- **Order** → purchase transaction
- **Payment** → Stripe metadata
- **CheckIn** → QR-based attendance
- **Category** → event classification

This clean separation enables:

- Analytics
- Refunds
- Attendance tracking

◆ Middleware Layer (middlewares/)

middlewares/

└─ auth.js

└─ error.js

auth.js

- JWT verification
- Protects private routes
- Injects req.user

error.js

- Centralized error handling
 - Prevents crashing on bad requests
-

◆ **Validation Layer (validators/)**

validators/

└─ index.js

- Request validation (likely Joi/Zod)
 - Prevents bad data from entering controllers
 - Ensures API safety
-

◆ **Utilities (utils/)**

utils/

├─ jwt.js

├─ logger.js

└─ qrcode.js

jwt.js

- Token creation & verification

logger.js

- Structured logs (Winston)
- Writes to logs/combined.log & error.log

qrcode.js

- Generates QR codes for tickets
 - Used in check-in flows
-

◆ Webhooks (webhooks/)

webhooks/

└─ stripe.webhook.js

- Handles Stripe events:
 - Payment succeeded
 - Payment failed
 - Updates orders/payments securely
 - Prevents frontend tampering
-

◆ Jobs & Services (Future-ready)

jobs/

services/

Intended for:

- Background tasks
- Email sending
- Scheduled cleanups
- Payment reconciliation

You've designed for **scalability from day one** 👍

🔗 Frontend Architecture (React + Vite + Tailwind)

frontend/

└─ index.html


```
└─ src/
  │ └─ api/
  │ └─ components/
  │ └─ pages/
  │ └─ layouts/
  │ └─ hooks/
  │ └─ store/
  │ └─ context/
  │ └─ utils/
  │ └─ App.jsx
  └─ main.jsx
```

◆ Entry Layer

index.html

- Root DOM container

main.jsx

- React bootstrap
- Redux provider
- Router provider

App.jsx

- Global route definitions
 - Layout composition
-

◆ API Layer (api/)

api/

└─ client.js

└─ auth.js

└─ events.js

└─ orders.js

client.js

- Axios instance
- Injects auth tokens
- Base URL config

Other files:

- Group API calls by domain
- Keep UI logic clean

◆ **Pages (pages/)**

pages/

└─ Home.jsx

└─ EventDetails.jsx

└─ Checkout.jsx

└─ Dashboard.jsx

└─ EditEvent.jsx

└─ Auth/

Pages represent **routes**.

Examples:

- / → Home
- /events/:id
- /checkout

- /dashboard

◆ Components (components/)

Reusable UI blocks:

components/

└─ Header.jsx

└─ Footer.jsx

└─ ProtectedRoute.jsx

└─ Dashboard/

ProtectedRoute.jsx

- Guards private routes
- Checks auth state

Dashboard/

- Organizer-focused UI
- Event creation
- Orders overview

◆ Layouts (layouts/)

layouts/

└─ Layout.jsx

- Shared page structure
- Header + Footer
- Keeps pages clean

◆ State Management (store/)

store/

└─ index.js

└─ slices/

Redux slices:

- authSlice.js
- eventSlice.js
- orderSlice.js

Each slice:

- Manages API calls
 - Stores loading/error states
 - Centralizes state
-

◆ Hooks (hooks/)

hooks/

└─ useApi.js

- Abstracts repetitive API logic
 - Cleaner components
-

5 How a Full Flow Works (Example)

🎫 Buy a Ticket

1. User clicks **Buy Ticket**
2. Frontend calls orders.js
3. Backend:
 - Creates Order
 - Initializes Stripe session
4. User pays on Stripe
5. Stripe → Webhook

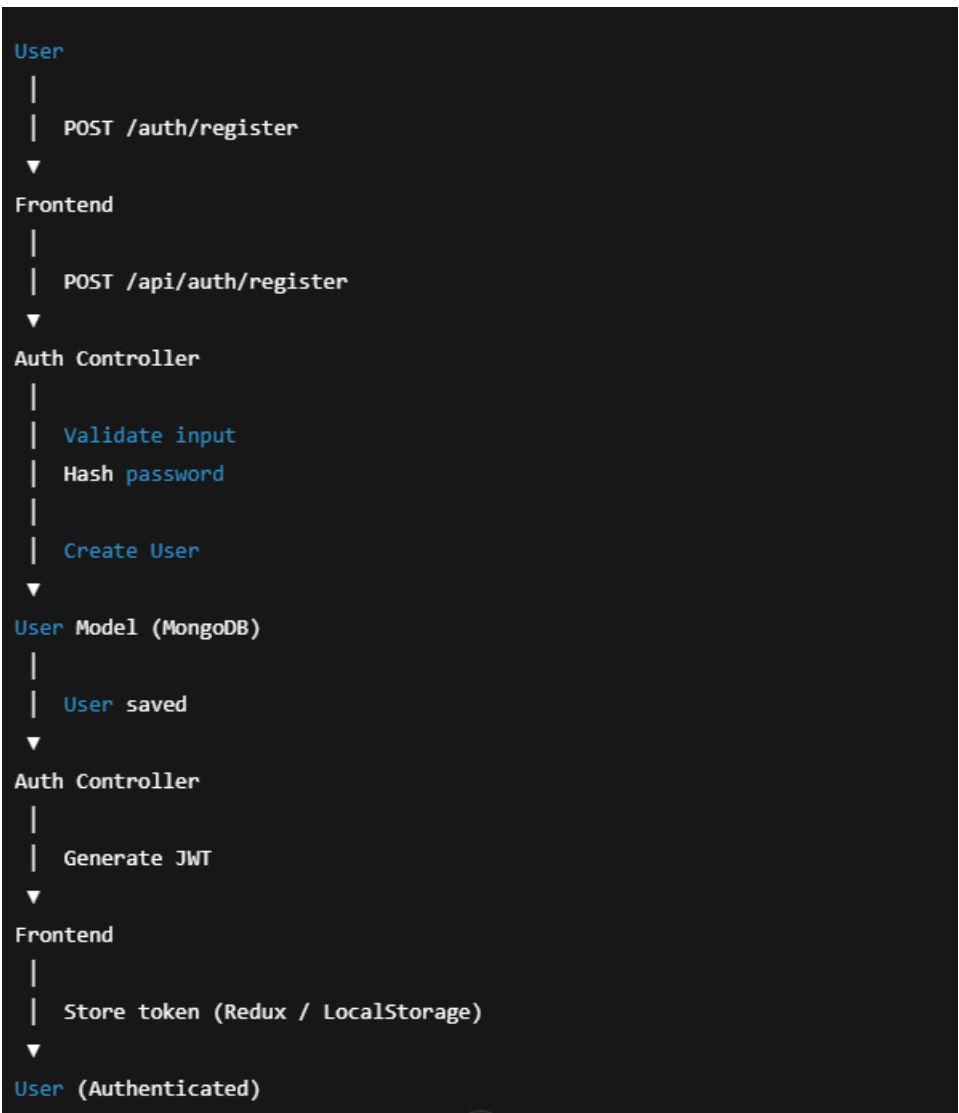
6. Backend:

- Confirms payment
- Issues Ticket
- Generates QR code

7. User sees ticket in Dashboard

1 Application Flow

1.1 User Registration & Authentication

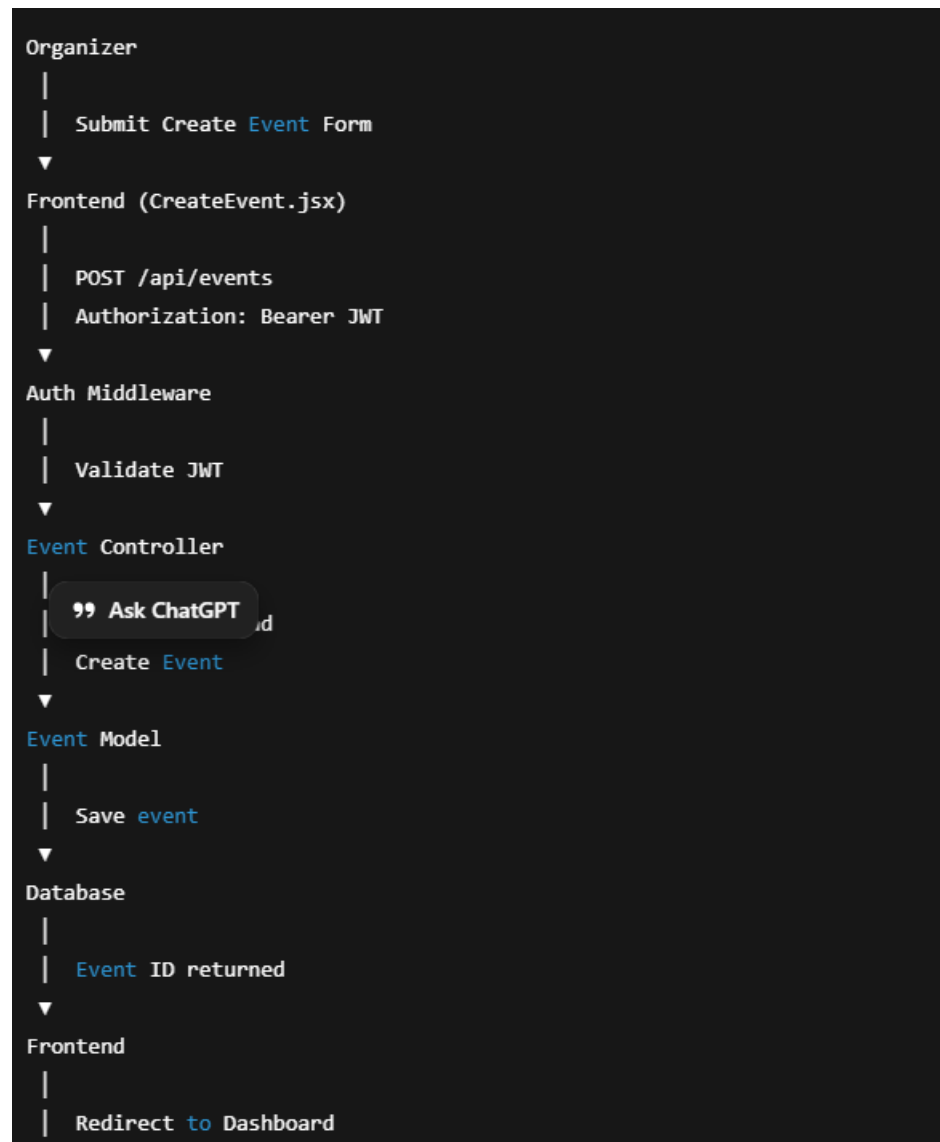


Key backend files involved

- auth.routes.js
- auth.controller.js
- User.model.js
- utils/jwt.js
- middlewares/auth.js

1.2 Event Creation (Organizer Flow)

Flow: Organizer creates an event

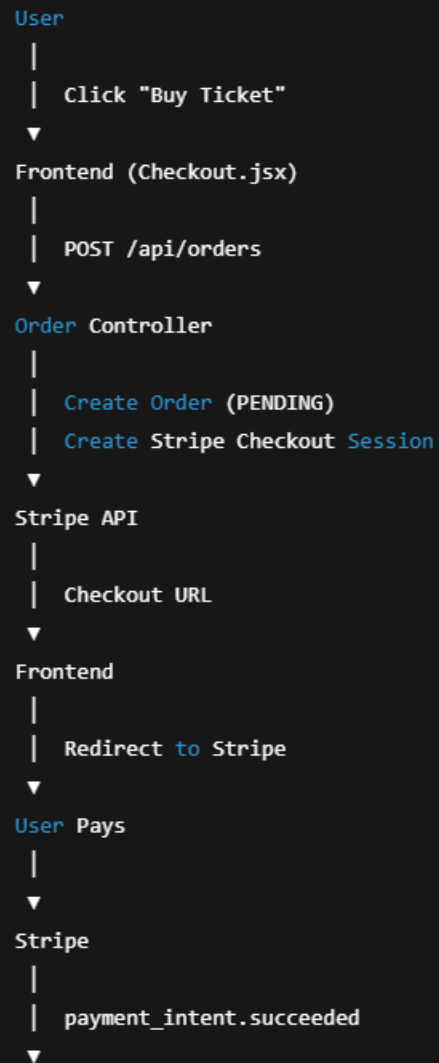


Key files

- event.routes.js
- event.controller.js
- Event.model.js
- validators/index.js

1.3 Ticket Purchase & Payment (Stripe)

Flow: Buy ticket → Pay → Confirm



```
Webhook Handler
|
| Verify signature
| Update Payment
| Mark Order PAID
| Generate Tickets
| Generate QR Codes
▼
```

```
Database
|
▼
```

```
User Dashboard
```

Key files

- order.controller.js
- Payment.model.js
- Order.model.js
- Ticket.model.js
- utils/qrcode.js
- stripe.webhook.js

1.4 Event Check-In (QR Code Scan)

Flow: Attendee check-in

```
Staff / Scanner App
```

```
|
| Scan QR Code
▼
```

```
Backend API
```

```
|
| Validate Ticket
▼
```

```
Ticket Model
```

```
|
| Check status
▼
```

```
CheckIn Model
```

```
|
| Create CheckIn record
▼
```

```
Backend
```

```
|
| Mark ticket as USED
▼
```

```
Response: SUCCESS / ALREADY USED
```


Key files

- `CheckIn.model.js`
- `Ticket.model.js`
- `ticket.controller.js`

2 ER Diagram (Entity Relationship Diagram)

This reflects your exact MongoDB models.

2.1 Core Entities

User

_id
name
email
password
role (organizer | attendee)
createdAt

Event

_id
title
description
location
date
organizerId (FK → User)
categoryId (FK → Category)
createdAt

Category

_id
Name

TicketType

_id
eventId (FK → Event)
name (VIP, Regular, Free)
price
quantity

Order

_id
userId (FK → User)
eventId (FK → Event)
totalAmount
status (PENDING, PAID, CANCELLED)
createdAt

Payment

_id
orderId (FK → Order)
stripePaymentIntentId
amount
currency
status

Ticket

_id
ticketTypeId (FK → TicketType)
userId (FK → User)
orderId (FK → Order)
qrCode
status (ACTIVE, USED)

CheckIn

_id
ticketId (FK → Ticket)
eventId (FK → Event)
checkedInAt

2.3 Relationship Summary

User 1 —< Event
User 1 —< Order
User 1 —< Ticket
Event 1 —< TicketType
Event 1 —< Order
Event 1 —< CheckIn

Order 1 —1 Payment
Order 1 —< Ticket
TicketType 1 —< Ticket
Ticket 1 —0..1 CheckIn